单卡 MoE 部署：七项相关工作的动机、洞察与实验证据

MECC 专家导览配套材料 | 2026-11-06

配套阅读：MECC 工作主页与一日讲义 · 17 页组会 PPT

这份材料回答七个问题：**每篇论文看到了什么现象？由此提出什么 Insight？怎样落成系统？具体用什么模型、卡、benchmark 和 baseline？实验结果到底是哪一个指标？** 各论文的数字均属于其原始实验配置；本文按论文分别解读，便于与 MECC 的单 A100、HDD 部署场景建立机制联系。

一眼看清实验地图

工作	设备	模型	Benchmark / 输入	Baseline	原文主要指标
SwapMoE	Jetson Nano、Jetson AGX Orin	SwitchT-16/32/64、GPTSAN	SAMSum 摘要；Wikipedia-Japanese 语言建模	原模型、Pruning、On-demand	内存、推理延迟、Rouge-2、PPL
Fiddler	Quadro RTX 6000 24 GiB；RTX 6000 Ada 约 48 GiB	Mixtral-8×7B，fp16	ShareGPT；附录扩展 ShareGPT/LMSYS	DeepSpeed-MII、Mixtral-Offloading、llama.cpp	tok/s、TTFT、beam search 速度
MoE-Infinity	单张 RTX A5000	Switch、NLLB、DeepSeek-V2-Lite、Mixtral、Arctic	BIGBench、FLAN、MMLU；长文本用 LongBench	DeepSpeed-Inference、Ollama/llama.cpp、Mixtral-Offloading、BrainStorm*	TPOT、长上下文延迟
MoE-Lightning	1×T4、1×L4、2—4×T4	Mixtral-8×7B、Mixtral-8×22B、DBRX	MTBench；HELM 合成推理和摘要	FlexGen、FlexGen(c)、DeepSpeed Zero-Inference	批处理总吞吐
SSD-LLaMA	1×RTX 5090 32 GB	DeepSeek-V4-Flash、Kimi-K2.7-Code、GLM-5.2，量化	MMLU、Alpaca	llama.cpp、KTransformers、Colibri	prefill/decode 速率、TTFT/TPOT
FlashMoE	1×RTX 5070 Ti 16 GB	OLMoE-1B-7B、Qwen3-30B-A3B	TriviaQA 训练/测试划分	LRU/LFU/LIFO/ARC/LeCaR；Fiddler、DAOP、llama.cpp、HF	缓存命中、tok/s、加载/prefill

工作	设备	模型	Benchmark / 输入	Baseline	原文主要指标
MoE-CORE	Ascend 950PR，84 GB NPU 限额	DeepSeek-V4-Flash-W4A8、GLM-5.2-W4A8C8	GSM8K、AIME 2026、HumanEval、MMLU、LongBench v2	Load-on-Demand、vLLM Prefetch	TTFT、TPOT、端到端延迟、TPS

表中 `BrainStorm*` 表示论文作者为评估自行扩展的机制版本。下面每节把“论文的 Insight”与“我们从中学到的系统问题”分开写，便于在组会讨论时准确定位贡献。

代表结果速览：

工作与场景	与谁比	原论文报告的结果
SwapMoE / SwitchT-16 摘要	原模型	峰值内存 4.08→1.64 GiB；延迟 0.82→0.45 s；Rouge-2 0.20→0.21
Fiddler / ShareGPT 普通单请求	最快的 llama.cpp 对照	平均生成速度 1.26×；beam search 对 llama.cpp 为 11.57×
MoE-Infinity / Mixtral	所测卸载系统	约 836 ms/token；论文描述约 1.4× 改善
MoE-Lightning / 单卡批处理	FlexGen	padding 版最高吞吐 3.5×；完整系统在所测配置中最高 10.3×
SSD-LLaMA / 三种量化 MoE	llama.cpp、KTransformers、Colibri 的适用对照	prefill 1.52—4.19×；decode 2.10—15.58×
FlashMoE / OLMoE	LRU 缓存策略	命中率约 +21%；生成速度约 +22%
MoE-CORE / DeepSeek GSM8K	Load on Demand	TPOT 66.5→38.0 ms/token；请求端到端 37.84→21.10 s

---

1. SwapMoE：让内存预算成为显式的设计输入

Motivation

MoE 的专家数量随模型规模增长，但边缘设备的可用内存很小。全部专家留在内存会放不下；每次按需读取会增加推理延迟；永久删去专家又会损失质量。SwapMoE 把问题表述为一个**预算内的内存—速度—质量权衡。**论文 §2

Insight 与方法机制

Insight：服务系统不必在每个时刻保留所有真实专家，可以维护一组重要的“虚拟专家”，并随输入变化更新它们与真实专家的对应关系。 系统先做细粒度专家画像，估计访问和质量影响；内存规划器决定不同层应保留多少虚拟专家以及更新频率；运行时根据输入调整映射。原论文采用遗传搜索寻找满足内存与延迟约束的配置。这样的设计把“哪些专家值得占据设备内存”从固定模型结构中解耦出来。论文 §3

原论文实验配置

项目	配置
设备	Jetson Nano、Jetson AGX Orin；batch=1，扫描不同内存和延迟预算
模型	Switch Transformer：每层 16/32/64 专家；GPTSAN
Benchmark	SAMSum 对话摘要，报告 Rouge-2；Wikipedia-Japanese 语言建模，报告 PPL
Baseline	原始 MoE；`Pruning`（按权重幅值保留部分专家且不交换）；`On demand`（缺失时再加载所需专家）
核心观测	内存、推理延迟、摘要质量或语言建模质量的三维关系

代表结果。 原文 SwitchT-16 的预算表给出：原模型峰值内存 **4.08 GiB**、延迟 **0.82 s**、Rouge-2 **0.20**；设置 2.0 GiB 预算后的虚拟专家方案使用 **1.64 GiB**、延迟 **0.45 s**、Rouge-2 **0.21**。另一组 SwitchT 摘要结果在显著降内存的同时出现小幅质量变化，展示了原论文追求的预算权衡。论文 §4.1—4.3、表 1

对 MECC 的启发。 预算应从任务真实可用资源出发，并允许逐层分配。MECC 的核心执行路径保留路由器选中的原专家，因此它关心的是完整权重的驻留与搬运，而不是 SwapMoE 的虚拟专家映射质量权衡。

2. Fiddler：一次专家未命中，也许应搬激活而非搬权重

Motivation

GPU 显存容不下完整 Mixtral 时，CPU 中的专家权重搬到 GPU 会经过 PCIe；对某些请求，把较小的激活发给 CPU 执行可能更快。但 CPU 计算能力和批处理效率与 GPU 差别很大，固定一种执行方式无法覆盖短请求、长 prefill、beam search。论文引言与 §3

Insight 与方法机制

Insight：专家执行位置应由“搬权重到 GPU 的总时间”与“在 CPU 上算并同步的总时间”共同决定。 Fiddler 根据工作负载和设备特征确定执行计划，并利用专家使用频率放置常用权重。长 prefill 往往使 GPU 计算更划算，batch 很小或生成方式不同则可能让 CPU 路径有优势。它的研究轴是**计算位置选择**，与缓存替换相关但不相同。论文 §3

原论文实验配置

项目	配置
环境 1	Quadro RTX 6000，24 GiB；Xeon Gold 6126，48 核；PCIe Gen3 ×16
环境 2	RTX 6000 Ada，约 48 GiB；Xeon Platinum 8480+，112 核；PCIe Gen4 ×16
模型	原始 fp16 Mixtral-8×7B；两环境最多分别让 56/256 与 125/256 个专家进入 GPU

项目	配置
Benchmark	ShareGPT 对话。普通单请求：输入 32/64/128/256、输出 64/128/256/512 token；长 prefill：输入 512/1024/2048/4096；beam search：输入 32、输出 64、beam width 4/8/12/16。附录另测 LMSYS-Chat-1M
Baseline	DeepSpeed-MII v0.2.3（启用 ZeRO-Infinity 和 pinned 内存）、Mixtral-Offloading（扩展为 fp16）、llama.cpp b2956
指标	单请求与 beam search 用生成 tok/s；长 prefill 用 TTFT

代表结果。 普通单请求在 15 组长度配置与两种设备上的平均速度，相对最佳基线 llama.cpp 为 **1.26×**；长 prefill 的平均 TTFT 相对 DeepSpeed-MII 改善 **1.07×**、相对 Mixtral-Offloading 改善 **1.65×**；beam search 相对 llama.cpp 的生成速度为 **11.57×**。三个加速比对应三种不同工作负载，不能合并为一个“Fiddler 总加速”。论文 §4、图 4—6

对 MECC 的启发。 专家交付不等于专家计算：HDD→CPU→GPU 的路径和 CPU 计算路径需要分别核算。MECC 现有 `fiddler` 实验行是 CPU 计算机制对照，便于观察不同执行位置对 TTFT 与 TPOT 的影响。

3. MoE-Infinity：看一条请求内部的激活模式

Motivation

个人机器上的 batch=1 推理与大规模在线服务不同：同一请求生成多个 token 时，专家激活常呈现稀疏的重复模式。只按最近使用时间替换，或只按全局频率预取，可能让下一层马上需要的专家被挤出缓存。论文 §4.1—4.2

Insight 与方法机制

Insight：一条请求的逐层专家激活轨迹，比全局热度更能提示接下来需要保留或预取什么。 MoE-Infinity 构建 request-level Expert Activation Matrix（EAM），从历史轨迹中挑选与当前请求相似的模式；再结合层的位置为缓存对象赋予优先级，并据此替换、预取。它在论文示例中解释了为什么传统 LRU 可能过早逐出下一层所需专家。论文 §4.5—4.7

原论文实验配置

项目	配置
GPU / 互连	单张 NVIDIA RTX A5000；PCIe 4.0，文中给出 32 GB/s 接口带宽
CPU 主存	按模型分别配置：Switch 64 GB、NLLB 256 GB、DeepSeek 32 GB、Mixtral 128 GB、Arctic 1 TB；模型参数可完整放入主存
模型	Switch-128×0.2B、NLLB-128×0.4B、DeepSeek-V2-Lite、Mixtral-8×7B、Arctic-128×4B
Benchmark	BIGBench 166 项、FLAN 66 项、MMLU 58 项，共 290 项任务；长上下文试验使用 LongBench
Baseline	DeepSpeed-Inference/FastGen、Ollama/llama.cpp、Mixtral-Offloading、作者扩展的 BrainStorm*；部分图还给出 vLLM

项目	配置
主协议	prompt 512 token、decode 32 token；报告平均每输出 token 延迟。长上下文另扫 4096 到 131072 token

代表结果。 Mixtral 在其配置下达到约 **836 ms/token**，相对若干支持卸载的对照约 **1.4×**；Switch、NLLB、DeepSeek 的论文报告值分别约 **155、531、155 ms/token**。论文摘要的 **3.1—16.7×** 是跨模型、任务和所测系统的范围，不是 Mixtral 单一对照的加速。论文 §5.1、图 7

对 MECC 的启发。 缓存可以使用比“专家总频次”更丰富的路由结构。MoE-Infinity 的主存配置能容纳完整模型；MECC 进一步研究任务主存不足、HDD 保留完整权重时的初始放置和在线驻留。

4. MoE-Lightning：高吞吐推理是调度问题

Motivation

离线批量生成时，模型权重交付、CPU 工作、GPU 工作如果串行发生，会浪费各设备的空闲时间。批大小、prompt 长度、输出长度又会改变瓶颈位置，固定调度难以持续接近高吞吐。论文 §3

Insight 与方法机制

Insight：通过 CPU/GPU/I/O 流水和性能模型一起选配置，可以让权重交付与计算彼此覆盖。 MoE-Lightning 的 CGOPipe 组织流水，paged weights 管理权重；Hierarchical Roofline Model（HRM）比较不同层级的带宽、算力与工作负载，以选择批大小、设备分工和交付策略。多卡时它还使用节点内张量并行。论文 §4

原论文实验配置

场景	模型与设备
S1	Mixtral-8×7B；1×T4 16 GB；24 核 Xeon、192 GB 主存
S2	Mixtral-8×7B；1×L4 24 GB；24 核 Xeon、192 GB 主存
S6/S7	Mixtral-8×22B；2×/4×T4；32 核 Xeon、416 GB 主存
S8/S9	DBRX；2×/4×T4；32 核 Xeon、416 GB 主存
Benchmark	MTBench：80 道多轮问题复制成批量负载，输出 32/64/128/256 token；HELM 合成推理和摘要
Baseline	FlexGen、启用 CPU attention 的 FlexGen(c)、DeepSpeed Zero-Inference v0.14.3；还比较自身 padding 变体
指标	生成 `token` 总数 ÷ (`prefill` + `decode` 总时间)，即批处理吞吐

代表结果。 原文在所测 MTBench/HELM 配置中报告最高 **10.3×** 吞吐改进。对于单 GPU 的 S1/S2，padding 版相对 FlexGen、FlexGen(c)、DeepSpeed 的最大改进分别为 **3.5×、5×、6.7×**；这是同一论文中更具体的一组系统对照。图 8 还比较了 2× 与 4×T4 的多卡规模。论文 §5.1—5.3

对 MECC 的启发。 缓存命中只解决权重“取几次”；流水决定取数时间“暴露多少”。面向单请求的 MECC 若将来扩展批处理，需要重新考虑动态工作集和流水调度，而不能把批吞吐与 TPOT 当成同一个指标。

5. SSD-LLaMA：将磁盘变成可执行专家的来源

Motivation

量化后的超大 MoE 仍可能超过消费级显存和主存。磁盘容量足够，但一次专家执行需要经过 I/O、主存保留和显存交付；单独优化某一段未必改善端到端速度。论文原文

Insight 与方法机制

Insight：磁盘 I/O 管线、三级驻留和 CPU/GPU 计算分工需要一起测量与协调。 SSD-LLaMA 按专家组织高效读取，动态管理磁盘—主存—显存中的权重，并按实际端到端时间选择专家在哪个设备执行。其论文所述主配置执行每个路由器选中的专家。对于 DeepSeek 平台，论文的 CPU/GPU 分工扫描发现“全部选中专家在 GPU 上算”比测试过的 CPU 分工更快，因此把 CPU 留给 I/O、缓存和调度。论文 §4.3—5

原论文实验配置

项目	配置
测试机	RTX 5090 32 GB、Intel Ultra5 230F、16 GB 主存、4 TB NVMe；PCIe 5.0 ×16，batch=1
模型	DeepSeek-V4-Flash：约 145.4 GiB，dense FP8 + 专家 MXFP4；Kimi-K2.7-Code：约 543.6 GiB、Q4_0；GLM-5.2：约 401.0 GiB、Q4_0
Benchmark	MMLU 和 Alpaca 请求；分别测 prefill 与 decode
Baseline	llama.cpp、KTransformers、Colibri；Colibri 只支持论文中 GLM-5.2 的那组对照
指标	prefill/decode token rate、TTFT、TPOT、读取带宽、每 token I/O 字节和显存缓存命中

代表结果。 摘要给出三模型家族、两类负载下的总体范围：prefill 对所测基线 **1.52—4.19×**，decode **2.10—15.58×**。正文在其 RTX 5090 + 32 GB 主存扩展配置上，报告 Kimi-K2.7-Code 达 **1.03 token/s**；默认 16 GB 主存配置用于主要比较。论文还给出“all-GPU 专家执行”与 CPU/GPU 分工的 DeepSeek 对照：相同读取工作量下，前者 TPOT 约 **514 ms**，把 1—6 个被选专家放到 CPU 后为 **530—574 ms**；加入 22 GiB 显存专家缓存后降至约 **149 ms**。论文 §4.3、§5

对 MECC 的启发。 这篇工作已经把完整专家交付、三级动态驻留和执行设备分工做成系统。MECC 的当前主线是任务可用主存较小、冷层为 HDD 时，逐层初始化和两级热缓存更新如何降低重复读取。两篇工作的机制有交集，适合逐项比较，而不是用系统名称直接判断胜负。

6. FlashMoE：最近性加频率，比单独 LRU 更会挑淘汰对象

Motivation

专家访问既有短期连续性，也有长期热度。LRU 只看最近一次，LFU 只看累计次数；在有限内存下，错误淘汰会反复触发磁盘读取。论文 §2—3

Insight 与方法机制

Insight：把最近性和频率作为特征，让轻量模型学习专家的未来重用距离，可直接优化缓存替换。 论文用 `1/recency` 和归一化 frequency 作为输入，以 Belady 最优替换产生训练标签，训练约 113 KB 的逐层 FFN。模型加载时只先装非专家权重，专家按需交付。训练路由轨迹来自 TriviaQA 的 512 条样本，每条最多 512 token。论文 §3、算法 1

原论文实验配置

项目	配置
推理测试机	RTX 5070 Ti 16 GB、Ryzen 5 9600X、64 GB 装机主存；实验锁定大部分内存，约留 1 GB 可用；PCIe 5.0 NVMe
训练/采样	TriviaQA 训练划分生成路由轨迹，测试题与训练题分开；论文给出单张 A100、batch=32 提取 OLMoE 路由轨迹的成本示例
模型	OLMoE-1B-7B、Qwen3-30B-A3B
Benchmark	TriviaQA 问答的模型装载 + prefill、decode 速度、缓存命中
缓存 baseline	LIFO、LFU、LRU、ARC、LeCaR；扫描每层缓存专家数
系统 baseline	Fiddler、DAOP、llama.cpp、Hugging Face Transformers

代表结果。 对 OLMoE，论文报告命中率相对 LRU 提高约 **21%**、相对 LFU 提高约 **51%**；相应 I/O 减少约 **22%** 和 **35%**。在相同系统条件下，学习式缓存相对 LRU 的生成速度改进约为 **22%（OLMoE）**、**7%（Qwen3）**。模型初始装载时间相对 llama.cpp 为 **4×**、相对 Fiddler/DAOP 为 **6.8×**；论文摘要的“最高 2.6×”来自跨系统推理比较，别把它理解成相对 LRU 的缓存策略加速。论文 §4、图 6—7

对 MECC 的启发。 “联合动态缓存”与“如何选被淘汰专家”是不同层次的设计。MECC 可以保留 GPU/CPU 升降级结构，再探索最近性、频率或学习信号；仓库中的 `flashmoe_rf_adaptation` 是公开特征的等权机制对照。

7. MoE-CORE：把逐层容量、画像和预取协同起来

Motivation

不同层的专家权重大小、路由热度和访问时序不同。统一按层分配相同缓存槽位，会让一些层闲置、另一些层频繁搬运；prefill 与 decode 的访存形态也不同。论文 §1—2

Insight 与方法机制

Insight：prefill 适合整层交替装载，decode 适合逐层非均匀驻留并预测下一层需求。 MoE-CORE 在 prefill 用两个交替缓冲重叠权重传输与计算；decode 结合任务领域画像生成初态、按层安排不同缓存容量、使用路由历史更新驻留，并跨层预取。主要对比运行路由器选中的原专家；论文另列可选的近似专家替换及多 token 预测实验。论文 §3

原论文实验配置

项目	配置
设备	Ascend 950PR；物理 NPU 内存 128 GB，实验对推理设置 84 GB 上限；主存上限 150 GB
模型	DeepSeek-V4-Flash-W4A8、GLM-5.2-W4A8C8
Benchmark	GSM8K、AIME 2026、HumanEval、MMLU、LongBench v2；每模型每数据集 200 请求、batch=1
Baseline	Load on Demand（LoD，解码阶段静态逐层缓存且缺失同步加载）；vLLM Prefetch（逐层预取）
输出协议	LoD 和 MoE-CORE 上限 1024 token；vLLM Prefetch 上限 128 token
指标	平均及 P90 TTFT/TPOT、端到端延迟、TPS；主要组件实验关闭近似替换

代表结果。 DeepSeek 的五类负载中，MoE-CORE 的平均 TPOT **38.0—44.8 ms**；相应 LoD 为 **62.1—70.6 ms**，即 **1.39—1.75×**。GSM8K 具体为 **66.5→38.0 ms/token**，端到端约 **37.84→21.10 s**。GLM 的平均 TPOT 为 **206.6—220.5 ms**，LoD 为 **229.1—244.8 ms**。论文还列出 vLLM Prefetch 的 TPOT，但其输出长度上限与另两项不同，适合阅读 TPOT，不适合直接用端到端请求时间求统一速度比。论文 §4、表 1

对 MECC 的启发。 逐层初始化、在线驻留和预取已是密切相关的设计轴。MECC 的重点是单 GPU、HDD 冷层、任务预算内的完整专家访问；讨论贡献时最有解释力的是相同权重、相同容量下逐项拆分初态、替换和数据交付的作用。

如何把七篇论文放进一次组会讨论

可以用同一个问题逐篇问：“**下一次路由选中一位目前不在 GPU 的专家，你让什么东西移动、让谁计算、又让哪个专家离开缓存？**”

工作	未命中时主要改变的决策	一句话 Insight
SwapMoE	有限内存里保留哪些虚拟专家	映射与预算规划共同决定质量、占用和速度
Fiddler	CPU 计算还是传权重到 GPU	决策应比较传输与计算的端到端成本
MoE-Infinity	该预取谁、该保留谁	请求级激活模式能提示后续层的访问
MoE-Lightning	何时传输与计算	流水减少设备等待，提高批吞吐
SSD-LLaMA	磁盘、主存、显存和执行设备如何配合	完整专家交付要端到端协调
FlashMoE	谁被淘汰	最近性与频率联合预测未来重用
MoE-CORE	每层留多少、何时预取	不同层和不同阶段需要不同的资源安排
MECC	部署前放谁、运行时 GPU/CPU 如何联动	初始热度与在线路由分别管理两个时间尺度

这也给出了阅读实验图的顺序：先看**模型与权重格式**，再看**GPU/CPU/磁盘预算**，接着看**benchmark和输出长度**，最后看论文到底报告的是 **TTFT、TPOT、tok/s、总吞吐、命中率还是质量。**只有这些条件相近，图中的高低才回答相近的问题。